Sparse hard oblique forests for jet classification

26 October 2026

Abstract

We study compact classifiers built from shallow oblique trees with linear leaves. Tree Alternating Optimization updates their cuts and leaf functions against a common forest loss. To reduce the number of coefficients, we select support with an L1 penalty, freeze it, and refit without shrinkage (Select–Freeze–Refit). In a jet-classification application, three depth-four trees with 418 nonzero coefficients achieve 56.20% signal efficiency at 1% background acceptance, compared with 55.48% for a 100-tree XGBoost reference with 3,099 scalar parameters. The results motivate sparse oblique forests as candidates for resource-constrained inference.

1 Introduction

Fast classification under hardware resource constraints motivates models that combine predictive accuracy with a small arithmetic workload. An axis-aligned decision tree tests one feature at each node; an oblique tree tests a linear combination. Oblique cuts can describe correlated decision boundaries within shallow trees. Linear leaf functions add first-order variation inside each region. Their coefficients also introduce a cost: a general affine function requires several multiplications and additions.

We investigate this tradeoff using additive forests of hard oblique trees. Tree Alternating Optimization (TAO) revisits the cuts of an existing tree, rather than fixing each cut in a single top-down construction [1]. Sparse oblique trees have also been used to analyze learned feature representations [2]. Here we optimize a shared classification loss for all forest members and study L1 regularization followed by fixed-support refitting. This separates coefficient selection from shrinkage, following the motivation of post-selection methods such as relaxed Lasso [3].

Our application distinguishes quark/gluon jets from W, Z and top jets using 16 high-level input features. The intended implementation context is FPGA inference for the CMS Phase-II Level-1 trigger. We compare floating-point classification performance, coefficient counts and arithmetic bounds, providing a basis for subsequent quantization and synthesis studies.

2 Oblique forests

2.1 Cuts, paths and leaf functions

An event is represented by a feature vector $x \in \mathbb{R}^F$ and a class label $y \in \{0, \dots, K-1\}$. A forest contains T binary trees of depth D, where depth counts decisions on a full root-to-leaf path. We label a node by its path p from the root: $p = \varnothing$ denotes the root, L its left child, R its right child, and RL the left child of R. Internal paths have length $|p| < D$; leaf paths have length D. Appending L or R identifies the two children. This notation describes the tree independently of storage.

At internal node p of tree $t \in \{1, \ldots, T\}$, an affine function

$$h_{tp}(x) = w_{tp}^{\mathsf{T}} x + b_{tp}, \qquad p \longrightarrow \begin{cases} pR, & h_{tp}(x) > 0, \\ pL, & h_{tp}(x) \leq 0, \end{cases} \tag{1}$$

determines the next branch. Here $w_{tp} \in \mathbb{R}^F$ is the cut's feature weight vector and $b_{tp} \in \mathbb{R}$ its bias. Routing is hard: each event follows exactly one branch. A cut is axis-aligned when only one component of w_{tp} is nonzero, and oblique when it combines several features.

A complete depth-D tree has $I = 2^D - 1$ internal nodes and $L_D = 2^D$ leaves. Its effective topology can be smaller. If $w_{tp} = 0$, the bias determines a constant route, and the other child and its descendants are unreachable for *every* input. For example, $w_{tp} = 0, b_{tp} = 0$ always chooses the left child by Eq. (1). Such branches can be removed without changing the function. A branch merely unused by a finite data sample is not necessarily unreachable.

Let $\ell_t(x)$ be the leaf path reached in tree t. With class $K - 1$ as a reference of zero logit, a linear leaf stores $K - 1$ affine functions

$$f_{t\ell k}(x) = a_{t\ell k}^{\mathsf{T}} x + c_{t\ell k}, \qquad k = 0, \ldots, K - 2, \tag{2}$$

where $a_{t\ell k} \in \mathbb{R}^F$ and $c_{t\ell k} \in \mathbb{R}$ are the leaf weights and biases. A constant leaf restricts $a_{t\ell k} = 0$. The forest logits and probabilities are

$$z_k(x) = \sum_{t=1}^{T} f_{t,\ell_t(x),k}(x), \quad z_{K-1}(x) = 0, \qquad P(y = k \mid x) = \frac{\exp z_k(x)}{\sum_{q=0}^{K-1} \exp z_q(x)}. \tag{3}$$

Thus tree logits are added before applying softmax. Linear leaves allow first-order dependence within a region, but do not enforce continuity across its boundaries. The forest is affine on each intersection of its trees' leaf regions; adding trees refines this partition.

For the binary application, class 0 is background and class 1 is signal. The signal margin and probability are

$$m(x) = -z_0(x), \qquad P(y = 1 \mid x) = \frac{1}{1 + \exp[-m(x)]}. \tag{4}$$

A larger margin therefore corresponds to a more signal-like event.

2.2 Parameter and arithmetic counts

Counting scalar weights and biases gives

$$P_{\text{dense}} = T\left[(2^D - 1)(F + 1) + 2^D B(K - 1)\right], \qquad B = \begin{cases} F + 1, & \text{linear leaves}, \\ 1, & \text{constant leaves}. \end{cases} \tag{5}$$

This is the allocated coefficient count for a complete topology. We separately report P_{nz}, the number of coefficients exactly different from zero, and P_{reach}, the nonzero count after removing algebraically unreachable branches. These counts include biases and exclude feature indices, connectivity and preprocessing metadata.

For $F = 16, K = 2, D = 4$, one linear-leaf tree has $15 \cdot 17 + 16 \cdot 17 = 527$ coefficient slots, while three have 1,581. A single dense tree may therefore have a similar *nonzero* count to a more strongly regularized forest of three trees, despite its smaller allocated count.

A dense binary forest with linear leaves needs at most $TF(D + 1)$ feature multiplications and TD comparisons along a selected path. Bias additions are not multiplications. Evaluating all internal functions and leaves instead uses $TF(2^{D+1} - 1)$ multiplications. Sparse functions reduce these counts according to their retained feature weights. They are arithmetic bounds; latency also depends on parallelism, multiplier reuse, precision and memory.

Implementation and reproducibility

Storage and inference. `HardTree` and `HardTAOClassifier` in `src/tao/hard.py` implement Eqs. (1)–(3). Internal nodes use heap indices $j = 0, \dots, I-1$, with children $2j+1, 2j+2$; leaf-array indices subtract I from the heap index. The arrays are `split_w`, `split_b` and `leaf_parameters`. The methods `routes`, `leaf_indices`, `predict_subtree`, `decision_function` and `predict_proba` implement traversal and logits. Input and logit arithmetic is binary64; public probabilities are cast to float32. The native ROC uses those probabilities, including any ties. Portable margin inference remains binary64. The current portable schema is binary, although the fitting code supports multiclass logits.
`sparsity_profile` in `src/tao/sparsity.py` computes exact-zero and reachable counts. The existing NumPy `dense` traversal may compute a node function over an entire batch before selecting reaching rows; its runtime does not represent a generated sparse path kernel.

3 Tree Alternating Optimization

3.1 A common forest objective

For training events (x_i, y_i), $i = 1, \dots, N$, let $v_i \geq 0$ be event weights and $W = \sum_i v_i > 0$. All results below use $v_i = 1$. We minimize

$$J(\theta) = \frac{1}{W} \sum_{i=1}^{N} v_i \, \mathrm{CE}(y_i, z(x_i)) + \lambda_s \sum_{t,p} \left(\|w_{tp}\|_1 + |b_{tp}| \right) + \lambda_\ell \sum_{t,\ell,k} \|a_{t\ell k}\|_1 , \qquad \mathrm{CE}(y, z) = \log \sum_{k=0}^{K-1} e^{z_k} - z_y. \tag{6}$$

Here θ denotes all cut and leaf parameters, $\lambda_s \geq 0$ penalizes cuts and $\lambda_\ell \geq 0$ penalizes leaf feature weights. Leaf biases are unpenalized; cut biases are penalized. For constant leaves, λ_ℓ has no effect. Every subproblem retains the full training-weight denominator W, so a penalty has the same normalization at different depths and node occupancies.

TAO alternates optimization blocks while other parameters are fixed [1]. An *epoch* visits every tree, updates its leaves and then its internal nodes from the bottom up, and finally refits all forest leaves jointly. Other trees contribute fixed logit offsets during each individual-tree update. Their offsets are refreshed before updating the next tree. Earlier trees remain trainable; the procedure does not freeze them as later trees are added.

3.2 Leaf optimization

With all routes fixed, the logits in Eq. (3) are linear in the leaf coefficients. The joint leaf problem is convex, although its parameters need not be unique. For a single tree, the sum of the other trees' logits is a fixed offset in the cross-entropy loss. A joint refit subsequently optimizes all trees' leaf functions against the same loss.

We use analytic cross-entropy gradients and a bound-constrained quasi-Newton solver. A penalized scalar coefficient is represented as $\theta_q = u_q - v_q$ with $u_q, v_q \geq 0$, contributing $\lambda(u_q + v_q)$; unpenalized coordinates are free real variables. After a finite solve, we reconstruct θ_q and evaluate the actual objective using $\lambda|\theta_q|$. This avoids reporting the larger auxiliary penalty when both u_q and v_q remain positive. A leaf update is retained only if its actual objective does not increase.

3.3 Decision-node optimization

Let $\mathcal{R}_{tp}$ be the training events reaching node p in tree t. Keep its ancestors, descendants and all other trees fixed. For event i, let $g_L(x_i)$ and $g_R(x_i)$ be the logit vectors predicted by its two subtrees, and o_i the logit sum from the other trees. The child losses are

$$C_i^L = \mathrm{CE}(y_i, o_i + g_L(x_i)), \quad C_i^R = \mathrm{CE}(y_i, o_i + g_R(x_i)), \quad \Delta_i = C_i^L - C_i^R. \tag{7}$$

A positive Δ_i favors the right child. Up to an additive constant, the cut subproblem is a weighted binary misrouting loss,

$$\frac{1}{W} \sum_{i \in \mathcal{R}_{tp}} v_i |\Delta_i| \mathbf{1}\{\mathbf{1}[h_{tp}(x_i) > 0] \neq \mathbf{1}[\Delta_i > 0]\} + \lambda_s(\|w_{tp}\|_1 + |b_{tp}|). \tag{8}$$

The indicator $\mathbf{1}[A]$ equals 1 when statement A holds and 0 otherwise. The reduced labels denote preferred *branches*, even for a multiclass classification loss. Events with $v_i|\Delta_i| = 0$ are uninformative for this cut and are excluded from its auxiliary fit.

We first fit a weighted L1-logistic classifier to the reduced labels with weights $v_i|\Delta_i|$. Its smooth loss produces a proposed cut, but is not Eq. (8). We therefore compare the existing cut, the logistic proposal, directions blending 25%, 50% or 75% of the proposal with the existing weight vector, and a constant direction. For each direction we sort its projected scores, move ties together, and search representable threshold intervals including the cut-bias penalty. We retain a strictly improving proposal under the actual hard-routing objective. This optimizes thresholds within the proposed directions, not over all possible hyperplanes.

During unpenalized fixed-support refitting, a cut that sends every positive-weight event to its preferred child already attains the pointwise lower bound of the reduced loss. We can certify it without solving an auxiliary logistic problem. When only one preferred branch is present, we skip the logistic fit and test allowed existing and constant directions.

3.4 Initialization and numerical stopping

Fresh trees are initialized by greedy axis-aligned classification trees fitted to bootstrap draws from the training sample. Subsequent TAO updates use the full training sample. Leaf class counts receive a 0.5 pseudocount before conversion to reference-class log odds; each tree's initial logits are divided by T. Early terminal nodes can be embedded in a complete topology using constant left-routing cuts, which explains some unreachable branches. The sparsity study below continues an existing trained forest.

We distinguish an optimizer iteration, an epoch of alternating tree updates, and convergence of the full run. Reaching an iteration cap alone is not convergence. For fixed routes, let g_q be the gradient of normalized unpenalized cross-entropy with respect to leaf coordinate θ_q, and let λ_q equal λ_ℓ for a penalized feature weight and zero for a bias. The leaf stationarity residual is

$$r_q = \begin{cases} 0, & q \text{ frozen}, \\ |g_q + \lambda_q \operatorname{sign}(\theta_q)|, & q \text{ free}, \ |\theta_q| > \delta, \\ \max(|g_q| - \lambda_q, 0), & q \text{ free}, \ |\theta_q| \leq \delta, \end{cases} \qquad r_{\mathrm{KKT}} = \max_q r_q, \tag{9}$$

with numerical zero tolerance $\delta = 10^{-10}$. Here KKT denotes the Karush–Kuhn–Tucker stationarity conditions. Frozen coordinates are subject to equality constraints and do not require zero gradients. The tolerance δ is used only for this residual; support selection uses exact zeros.

We require $r_{\mathrm{KKT}} \leq 10^{-6}$, successful inner solves, relative objective gain at most 10^{-5} and a routing-change fraction at most 10^{-4} for three consecutive epochs, after at least five epochs. An event counts as rerouted if its leaf changes in any forest member. The relative gain is

$\max(0, (J_{\text{before}} - J_{\text{after}})/\max(|J_{\text{before}}|, 10^{-12}))$; increases above 10^{-8} are rejected. These conditions concern numerical stationarity of leaf fits and the available cut proposals. They do not establish global optimality of the forest.

Implementation and reproducibility

Objective and leaf solvers. `src/tao/hard.py` contains `cross_entropy_rows`, `ce_value_gradient`, `fit_leaf_parameters`, `HardTAOClassifier._penalty`, `objective`, `_joint_design` and `global_refit`. The joint CSR design activates a leaf block in each tree: a linear-leaf row has $T(F+1)$ stored entries. For 531,200 events and three trees this is 27,091,200 entries. Frozen feature columns are still stored. The solver is SciPy L-BFGS-B. `fit_l1_split` also supports scikit-learn LIBLINEAR for ordinary penalized proposals, with $C = 1/(\lambda_s W)$ and a penalized synthetic intercept. Zero-penalty and masked proposals use L-BFGS-B.
Cuts and epochs. `best_threshold` in `src/tao/split_search.py` implements the threshold search; `FrozenSupportTAO._split_update` in `src/tao/support.py` handles fixed support and pointwise certificates. `begin_epoch`/`step_epoch` in `src/tao/epoch.py` serialize a bottom-up epoch and cache other-tree logits. `initialize_cart` in `src/tao/initialization.py` initializes fresh trees. `append_zero_tree` can extend a forest while preserving predictions; the selected compact forest was not obtained with that option. `leaf_kkt_residual` and `convergence_check` in `src/tao/convergence.py` implement Eq. (9) and the stopping tests.

4 Selecting support and refitting

4.1 Select–Freeze–Refit

L1 regularization both removes coefficients and shrinks surviving values. Once a coefficient will be retained, shrinkage can unnecessarily reduce accuracy. Motivated by post-selection refitting [3], we use the following procedure, termed Select–Freeze–Refit (SFR):

1. **Select.** Follow split-only, leaf-only or joint L1 paths using converged terminal fits as warm starts. Choose a support strategy on validation.
2. **Freeze.** Copy the selected parent, fix its topology, and record $M_q = \mathbf{1}[\theta_q \neq 0]$ for every coefficient, including biases.
3. **Refit.** Remove both L1 penalties. First optimize the surviving leaf coefficients with cuts fixed; then optionally alternate cut and leaf updates while preserving the original masks.

Let $\mathcal{C}$ denote coordinates of input-independent cuts and $\theta_{\mathcal{C}}^{\text{parent}}$ their parent values. The full refit minimizes

$$\frac{1}{W}\sum_i v_i \,\mathrm{CE}(y_i, z(x_i;\theta)) \quad \text{subject to } \theta_q = 0 \text{ if } M_q = 0, \quad \theta_{\mathcal{C}} = \theta_{\mathcal{C}}^{\text{parent}}. \tag{10}$$

Free coefficients may change sign or pass through zero; their masks are not reselected. Frozen-zero coefficients cannot regrow. Cuts that were already input-independent retain their parameters and constant decision. A frozen-zero cut bias also remains zero during threshold search. This differs from simply lowering L1 on an unrestricted forest, which could restore removed parameters.

Each stage begins with a joint fixed-route leaf fit; the full stage repeats it after each epoch of cut updates. Model selection uses the declared validation working point among eligible saved fits. A run that eventually converges can have its best validation score at an earlier epoch; the selected snapshot need not itself be stationary.

4.2 Separation and cut-scale ambiguity

Unpenalized logistic fits can exhibit complete or quasi separation, with the loss approaching an infimum as coefficients diverge. With routes fixed, logits are linear in free leaf coordinates. We search for a free direction d such that every true-class margin against another class has a nonnegative change and at least one has a positive change:

$$\frac{d}{d\alpha}\left[z_{y_i}(x_i;\theta+\alpha d)-z_k(x_i;\theta+\alpha d)\right]\geq 0 \quad (v_i>0,\ k\neq y_i). \tag{11}$$

These are linear inequalities in d. A box-constrained linear program fixes the scale of the direction and searches for a positive summed improvement. A positive feasible result indicates a separating direction; an ambiguous result or timeout is held as inconclusive. Such stages are excluded from selection, while a separately completed leaf-only stage can remain eligible. This diagnostic does not establish uniqueness of the fitted coefficients.

Cut scale presents a separate issue. For any $c>0$, in exact arithmetic,

$$\mathbf{1}[w^{\mathsf{T}}x+b>0]=\mathbf{1}[cw^{\mathsf{T}}x+cb>0], \qquad \|cw\|_1+|cb|=c(\|w\|_1+|b|). \tag{12}$$

The hard-routing loss is unchanged by positive rescaling, while raw-coefficient L1 decreases as c decreases. The logistic proposal fixes a scale within its smooth subproblem, but the outer hard objective remains scale-ambiguous. Thus L1 is an empirical sparsity control in this alternating procedure, not an invariant measure of tree size. Small nonzero cut coefficients cannot simply be discarded: their ratios may still define an important boundary. Explicit rescaling before quantization requires verification near boundaries. An input-independent cut can instead be replaced by its Boolean branch choice.

Implementation and reproducibility

Masks and separation. `freeze_support`, `continue_full_refit`, `FrozenSupportTAO` and `separation_check` are in `src/tao/support.py`. They check topology, shapes, finite values, mask hashes and absence of zero regrowth. Leaf-only cuts remain bit-identical. The separation LP uses all positive-weight training events, checks feasibility, and treats scores above 10^{-7} as positive and at or below 10^{-10} as near-zero; intermediate values and 600-second timeouts are held. Certificates are cached only while routes remain unchanged.
A concrete precision trap occurs in the compact model: scientific node $(t,p)=(2,RR)$ (storage tree 1, node 6) has zero weights and bias $2^{-1074}>0$. It always chooses right. Rounding the bias to zero changes the decision; folding the constant branch preserves it without representing a subnormal constant in fixed-point arithmetic.

5 Jet-classification study

5.1 Data and comparison protocol

We use 16 high-level jet features from the dataset catalogued as OpenML 42468 [4]. Class 0 combines quark and gluon jets; class 1 combines W, Z and top jets. The fixed partitions contain 531,200 training, 132,800 validation and 166,000 test events, using split seed 42. Validation contains 52,511 background and 80,289 signal events. Each feature is standardized by its training mean μ_f and standard deviation $s_f>0$:

$$x_f=(r_f-\mu_f)/s_f, \qquad f=1,\dots,F, \tag{13}$$

where r_f denotes the unstandardized feature. All models use the same partitions, labels and feature order.

The primary metric is signal efficiency ϵ_s at background efficiency $\epsilon_b = 0.01$, interpolated on the empirical receiver operating characteristic (ROC). AUC denotes its area. Both use the predicted signal probability. For a threshold τ, the operating efficiencies are the fractions of signal and background events accepted by a score above τ. Interpolation can give a point not exactly realized by a strict threshold. With the validation sample here, a realizable threshold accepting at most 1% background permits at most $\lfloor 0.01 \cdot 52{,}511 \rfloor = 525$ background events.

All oblique scans, SFR choices and results below use validation, with AUC secondary to the low-background working point. The test partition was used in an earlier XGBoost reproduction check, but not to select the oblique models or penalties. Reported uncertainties consequently concern validation and condition on the selected fitted models.

The XGBoost reference is an additive ensemble of axis-aligned trees with constant leaves, trained using gradient/Hessian information for greedy splits and regularized leaf updates [5]. It has 100 trees, maximum depth 4, learning rate 0.1, row and feature subsampling 0.8, histogram-based splits, binary logistic objective and random seed 42. Its realized structure contains 1,499 thresholds, 1,599 leaf values and one base bias: 3,099 scalar parameters. Feature indices and topology are excluded, consistently with the oblique count.

Implementation and reproducibility

The project repository [6] contains the archived data contract and training configuration `configs/bdt_qg_vs_wzt_seed42.yaml`; `src/train_bdt.py` trains the reference. The input files are `data/qg_vs_wzt_standardized.npz` and `data/qg_vs_wzt_metadata.json`. `scripts/audit_tao_splits.py` checks split identities. The data hash is

`f5ec6f7ed442010656a904700f8f6421cc34b674a1ec810c5cf6b2de003ed926`

No new split is drawn from OpenML. The root README documents the historical XGBoost test reproduction. `src/evaluate.py` supplies the common ROC interpolation. XGBoost version 3.2.0 is used with one thread; remaining settings are those of that version. The handed-over native JSON fixes the actual model. The paper input generator evaluates validation only.

5.2 Selected models and fixed-support study

Table 1 compares three oblique configurations and XGBoost. All oblique entries use depth-four trees and linear leaves. We call the 418-coefficient three-tree model *Compact refit*; the 837-coefficient model is the *higher-efficiency* alternative. A one-tree reference illustrates the distinction between allocated and retained parameters. Figure 1 shows the corresponding ROCs.

Table 1: Validation comparison. Layout gives tree count and maximum depth (D4 means depth four). Dense and nonzero counts include biases. CPU is median bulk time per event in microseconds, measured as described in the text.

Model	Layout	P_{dense}	P_{nz}	ϵ_s [%]	AUC	CPU [μs]
Compact refit	3 × D4	1581	418	56.20	0.92733	0.974
Higher efficiency	3 × D4	1581	837	57.12	0.92967	1.131
One-tree reference	1 × D4	527	452	55.63	0.92704	0.412
XGBoost	100 × D4	3099	—	55.48	0.92056	1.083

The Compact refit uses 49% fewer allocated scalars than XGBoost and about half the nonzero coefficients of the higher-efficiency oblique model. These are different comparisons: a coefficient slot is not necessarily nonzero, and an oblique feature weight does not have the same hardware cost as an axis-aligned threshold. The one-tree reference has fewer allocated slots but more nonzero

coefficients than Compact refit.

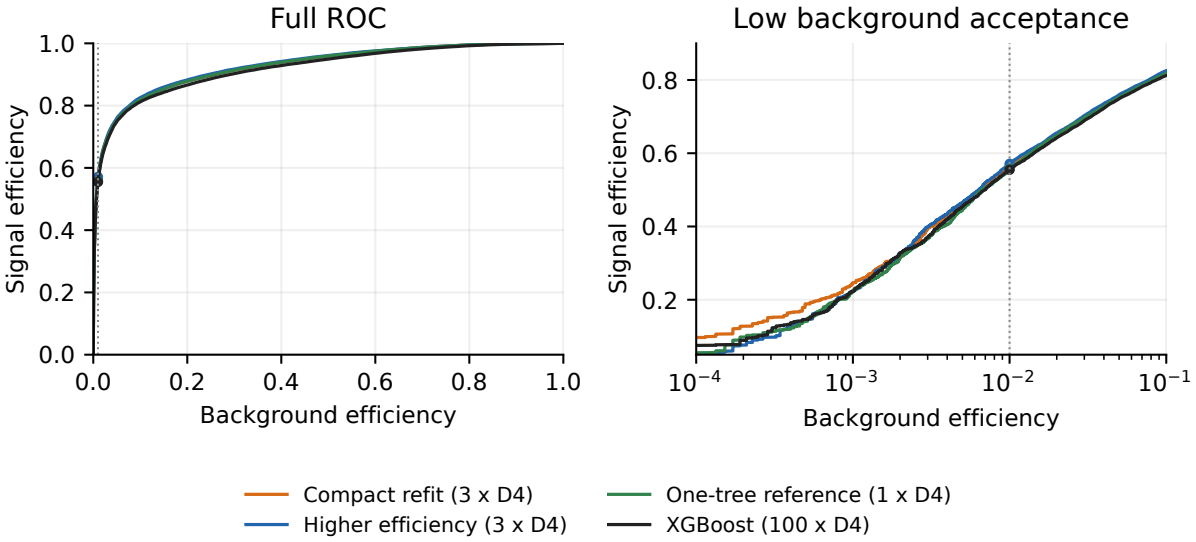


Figure 1: Validation ROCs of the four selected models. The right panel resolves low background acceptance; dots mark the interpolated 1% background point.

For Compact refit, a converged three-tree parent was continued along a split-penalty path $\lambda_s = 3{\cdot}10^{-5}, 10^{-4}, 3{\cdot}10^{-4}, 10^{-3}, 3{\cdot}10^{-3}$, with $\lambda_\ell = 10^{-5}$. The terminal fit at the final penalty supplied 418 coefficients. Leaf-only refitting raised validation efficiency from 54.59% to 54.81%. Full alternating refitting converged after 15 epochs at 55.98%; its best saved validation snapshot was epoch 8 at 56.20%. The selected model is this earlier snapshot, with the same 418 coefficients. The one-tree reference was trained without L1 and without SFR.

Table 2 and Fig. 2 summarize ten parent supports. Six full refits completed, two were held for separation and two for an inconclusive linear-program timeout. The higher-efficiency model is a completed leaf-only refit at 57.12%, improving its own parent by about 0.21 percentage points. Its full stage was held and is excluded from selection. Stronger penalization does not consistently improve the accuracy–size tradeoff; refitting cannot restore a direction removed by support selection.

Table 2: Fixed-support refitting study. Efficiencies are percentages at 1% background. All models have three depth-four trees except the starred one-tree case. Parent and leaf columns are terminal stage results; best is the best eligible saved leaf/full refit. Held full stages contribute no candidates. The support budget is the parent's exact nonzero count.

λ_s/λ_ℓ	Support	Parent	Leaf	Best	Full stage
0.0001/0.0001	520	55.58	55.82	56.49	complete
0.0003/1e-5	675	56.98	56.74	56.74	timeout
1e-5/3e-5	837	56.91	57.12	57.12	timeout
0.0003/0.0003	394	54.48	54.85	54.85	separated
1e-5/1e-5*	403	55.20	55.06	55.08	complete
0.001/1e-5	561	56.06	56.32	56.46	complete
3e-5/3e-5	777	56.62	56.75	56.75	complete
1e-5/1e-5	889	56.40	56.45	56.74	complete
0.003/1e-5	418	54.59	54.81	56.20	complete
0.001/0.001	317	51.27	52.52	52.52	separated

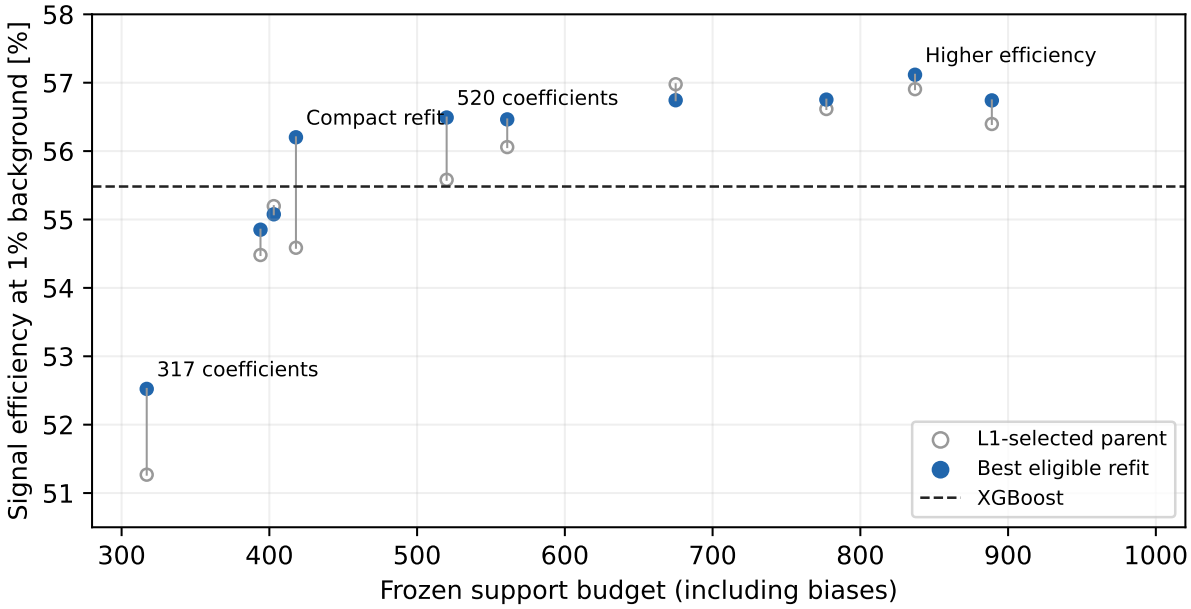


Figure 2: L1-selected terminal parents and their best eligible fixed-support refits. Each vertical segment keeps the support budget fixed. Only refits from eligible stages contribute; the penalty settings are given in Table 2.

5.3 Uncertainty and inference cost

A paired class-stratified bootstrap with 1,000 replicates, seed 20261025, recomputes the interpolated working point in each replicate. Table 3 gives percentile intervals for the difference from XGBoost. The Compact interval includes zero. These intervals condition on the selected models and account for neither hyperparameter/checkpoint selection nor training-seed variation; they are not independent test claims.

Table 3: Conditional validation efficiency difference from XGBoost, in percentage points (pp), with paired 95% percentile intervals.

Model	Difference [pp]	Interval [pp]
Compact refit	+0.72	[-0.43, +1.52]
Higher efficiency	+1.63	[+0.65, +2.55]
One-tree reference	+0.15	[-1.08, +0.97]

Table 4 gives sparse arithmetic bounds. For comparison, a dense three-tree depth-four linear forest has 240 selected-path or 1,488 all-node feature multiplications and 12 path decisions. The CPU timings in Table 1 use 132,800-event validation batches, one thread, nine repeats and the median wall time per event. They compare a NumPy oblique traversal with compiled XGBoost and are bulk-throughput measurements. Parameter and operation counts motivate implementation studies; FPGA cost requires a numeric format, device, clock and synthesis strategy.

Table 4: Sparse arithmetic bounds after removing input-independent unreachable branches. Path counts bound one selected route; all-node counts evaluate every reachable affine function. Bias additions are excluded from multiplications.

Model	P_{reach}	Path mult.	All-node mult.	Path decisions
Compact refit	379	80	327	12
Higher efficiency	837	163	744	12
One-tree reference	452	65	421	4

Implementation and reproducibility

Selected results. The frozen comparison uses the v3 model bundle. `fpga_handoff/CURRENT_MODEL.json` selects `fpga_handoff/v3/compact_refit_3t_d4/`. The 837-coefficient alternative is `highest_efficiency_3t_d4`; the one-tree reference is `single_tree_1t_d4`. All exact source snapshots and masks are recorded in the handover manifest. A later 57.24% full-refit diagnostic for the higher-efficiency support is held and not an eligible model.

Compact training chain. Start from the v2 `winner_3t_d4` coefficients, run `configs/l1_path_v1/control.json`, then the split path `split_01.json` through `split_05.json` in `configs/l1_path_v1/`. The manifest is `configs/tao_l1_path_v1.json`. The final terminal parent feeds `configs/support_refit_scan_v1/split300.json`. Parent training uses `scripts/train_tao_fully.py`; SFR uses `scripts/train_tao_support_refit.py`, `scripts/run_support_refit_scan_case.py` and `scripts/support_refit_artifacts.py`. Exact refit reproduction begins with the archived hash-pinned terminal parent, rather than a fresh fit at the final penalties. The selected checkpoint hash is

`e89b8923fba66f5df02046e04a1828a8ea4c56f60c1d7b201a2a2f92a0ab00fa`

Budgets. The penalty path starts with local/joint/split iteration budgets 300/250/300 and logs repeated chunks or adaptive increases. Compact refit uses 400/400/800, L-BFGS-B tolerances `ftol=1e-12`, `gtol=1e-8`, at most 100 joint-fit chunks and 200 epochs per stage. A 2,700-second allocation checkpoints with return code 75. A mild-control router required a documented 800-to-4,000 iteration-budget amendment and converged after 2,028 auxiliary iterations; this changed no objective or stopping tolerances. See `scripts/amend_support_solver_budget.py` and `docs/support_refit_scan_budget_amendments.json`.

Research status. These results were frozen on 25 October 2026. The separate dated record `docs/handover_status_2026-10-26.json` found 37/58 architecture cases converged and the 16-point L1 path complete. A running five-tree depth-four checkpoint reached 57.13% with 1,258 nonzeros but was unconverged and was not promoted. `src/tao/uncertainty.py` implements the bootstrap; `scripts/build_paper_inputs.py` regenerates frozen tables and curves. `sources/provenance.json` in this document folder records hashes of inputs, numerical source files, tables and figures.

6 An example learned tree

Actual cuts illustrate the mixture of axis-aligned, oblique and constant nodes. In Compact refit, using the standardized feature indices of Table 5, the first tree contains

$$h_{1,\varnothing}(x) \simeq x_4 + 0.4216204 \tag{14}$$

$$h_{1,L}(x) = 0 \tag{15}$$

$$\begin{aligned} h_{1,R}(x) \simeq\ & 0.088106595x_4 + 0.0025365055x_6 + 0.17735909x_{11} \\ & + 0.031223362x_{15} + 0.10064081x_{16} + 0.019058664 \end{aligned} \tag{16}$$

Coefficients marked $\simeq$ are rounded to eight significant digits. At the root an event goes right if $x_4 > -0.4216204$. Otherwise it reaches L, whose constant cut sends it left again. Hence the entire LR subtree is unreachable. The root's right child combines five input features.

For the second tree, another learned cut is

$$h_{2,L}(x) \simeq -2.3776136 \cdot 10^{-11} x_{16} - 3.1703281 \cdot 10^{-11} \tag{17}$$

Its small scale does not make it uninformative: it routes right approximately when $x_{16} < -1.3334$. Setting both coefficients to zero would turn it into a constant left route. This example motivates distinguishing exact sparsity from coefficient magnitude before quantization.

Implementation and reproducibility

Print the actual functions. `TreePrinter` in `src/tao/printing.py` provides `print_tree`, `format_tree`, `format_forest` and `records`. The CLI `scripts/print_tao_tree.py` prints every cut and affine leaf, including unreachable branches. It is read-only and separate from pinned fitting code. For example, after setting the environment described below:

```
import json
from pathlib import Path
from tao.printing import TreePrinter
spec = json.loads(Path(
    "fpga_handoff/v3/compact_refit_3t_d4/standardized/model.json"
).read_text())
TreePrinter(spec, precision=17).print_tree(0)
```

The complete 17-digit record is `generated/compact_forest_full_precision.txt`; the annotated supplement prints all three trees at nine display digits. Only exact-zero terms are omitted. Independent tests parse the printed expressions, recover all 418 nonzero coefficients, count 379 reachable coefficients, and compare predictions with the portable evaluator. They also check equality routing, constant leaves, tiny terms and non-mutation. The full suite passed 81 tests.

7 Discussion

Shallow oblique forests with linear leaves provide competitive validation performance with fewer coefficients than the axis-aligned reference in this application. Compact refit reaches 56.20% signal efficiency at 1% background acceptance, with 418 nonzeros in 1,581 allocated slots. The higher-efficiency alternative reaches 57.12% with 837 nonzeros. The conditional uncertainty on the Compact–XGBoost difference includes zero, so the principal result is a compression point at comparable validation performance.

Fixed-support refitting recovers accuracy lost to shrinkage without restoring removed coefficients. Its benefit depends on the selected support: aggressive L1 can remove useful directions that refitting cannot recover. Separation, cut-scale ambiguity and the distinction between a converged terminal model and a validation-selected earlier snapshot remain relevant to interpreting training results. Axis-aligned cuts are contained in the oblique family, but this does not imply that three shallow trees can represent every function of a hundred-tree ensemble.

The next step is to quantify how the reduced arithmetic workload translates into hardware. This requires explicit feature, coefficient and accumulator formats, verification of routing changes near cut boundaries, calibration of a realizable working-point threshold, and synthesis measurements at a specified device and clock. Further model choices should be completed before additional test evaluation, retaining disclosure of the earlier XGBoost-only test check.

Implementation and reproducibility

Environment and entry points. Training and input regeneration use Python 3.10 with the pinned `requirements.txt`: NumPy 1.26.4, SciPy 1.15.2, scikit-learn 1.7.2, XGBoost 3.2.0, PyYAML 6.0.3 and matplotlib 3.10.7. The reference CLIP checkout is `/users/robert.schoefbeck/CMS/tree_benchmark`; its Conda environment is `tree-benchmark`. On CLIP use the conda-forge CPU XGBoost build, since the PyPI Linux wheel requires newer glibc than the reference login environment. No GPU is needed.

```
source /software/f2022/software/mamba/24.3.0-0/etc/profile.d/conda.sh
conda activate tree-benchmark
cd /users/robert.schoefbeck/CMS/tree_benchmark
export PYTHONPATH="$PWD/src:$PWD" PYTHONDONTWRITEBYTECODE=1
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1
python scripts/print_tao_tree.py \
  --model fpga_handoff/v3/compact_refit_3t_d4/standardized/model.json \
  --precision 17 --output /tmp/compact_forest.txt
```

For a new training account or checkout, the complete environment-creation and controlled refit recipe is in `docs/DORIAN_HANDOVER.md`. The datasets and pickle archives are deliberately not in Git. Exact reproduction of the refit starts from the hash-pinned terminal parent; rebuilding its entire L1 path can produce differently serialized checkpoint bytes and requires a new explicit source hash/run name. Existing named runs must not be resumed under changed training code or stopping policies. The standalone `paper/` folder, however, contains all generated text/tables and figures needed to compile this PDF without datasets, model pickles or a Python training environment.

Implementation map.

Object / operation	Entry point
Routing, logits, objective	`HardTree`; `HardTAOClassifier` in `src/tao/hard.py`
Weighted leaf optimization	`fit_leaf_parameters`; `ce_value_gradient`
Joint leaf design/refit	`_joint_design`; `global_refit`
Reduced split proposals	`_hard_split_update`; `FrozenSupportTAO._split_update`
Sorted hard threshold search	`best_threshold` in `src/tao/split_search.py`
Node-resumable epoch	`begin_epoch`; `step_epoch` in `src/tao/epoch.py`
Masks and separation	`freeze_support`; `continue_full_refit`; `separation_check`
Stopping and exact-zero work counts	`leaf_kkt_residual`; `convergence_check`; `sparsity_profile`
Read-only equations	`TreePrinter.print_tree` in `src/tao/printing.py`
Portable inference / firmware export	`export_model`; `predict_margin`; `cpp_header` in `src/tao/portable.py`

Frozen coefficients and reference verification. `fpga_handoff/v3/manifest.json` hashes the candidate JSONs, hexadecimal C++ headers, feature contract, conservative training-box ranges and golden vectors. Each candidate has both standardized and raw-input variants; choose one and use its matching vectors and working-point threshold. The default JSON hash is

`0ce4fabc4ca74d3fcd2ddb8c94bb6b614b4f2fb998a9225873e167977922d62a`

The corresponding immutable pickle is named by the checkpoint hash given in the Compact training chain below, under `models/support_refit_scan_v1_selected/` on CLIP. The binary portable JSON and headers suffice for firmware; Python pickles are not needed there.

The C++17 reference in `fpga_handoff/reference.cpp` passed all six v3 variants: 1,594 vector evaluations with exact leaf assignments and maximum margin difference below 2.5×10^{-14}. The verifier `scripts/verify_fpga_handoff.py` needs only Python's standard library and a C++17 compiler. It checks bundle hashes before compiling/running the golden vectors and preserves an existing frozen verification report. Fixed-point verification remains a separate next

step. v1 and v2 are retained historical bundles; `fpga_handoff/CURRENT_MODEL.json` selects v3 Compact refit for new implementation work.

References

[1] Miguel Á. Carreira-Perpiñán and Pooya Tavallali. Alternating optimization of decision trees, with application to learning sparse oblique trees. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/hash/185c29dc24325934ee377cfda20e414c-Abstract.html.

[2] Suryabhan Singh Hada, Miguel Á. Carreira-Perpiñán, and Arman Zharmagambetov. Sparse oblique decision trees: A tool to understand and manipulate neural net features. arXiv:2104.02922, 2021. URL https://arxiv.org/abs/2104.02922. First posted in 2021; revised manuscript and journal version in 2023.

[3] Nicolai Meinshausen. Relaxed lasso. *Computational Statistics & Data Analysis*, 52(1):374–393, 2007. doi: 10.1016/j.csda.2006.12.019. URL https://people.math.ethz.ch/~nicolai/relaxo.pdf.

[4] OpenML. hls4ml_lhc_jets_hlf, dataset 42468. Dataset catalogue. URL https://www.openml.org/d/42468. The project uses its archived feature arrays and fixed splits, rather than a new split from the catalogue.

[5] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785. URL https://arxiv.org/abs/1603.02754.

[6] HEPHY Analysis Software. Oblique tree benchmark, 2026. URL https://github.com/HephyAnalysisSW/oblique-tree-benchmark. Implementation, frozen v3 FPGA handover and dated research records; exact source fingerprints accompany this note.

A Input feature order

Table 5 defines the coordinates r_f of the archived feature vector, using the source dataset's feature names. All displayed cuts use their standardized counterparts x_f from Eq. (13).

Table 5: Feature-coordinate convention. Names identify columns in the high-level jet dataset [4].

f	Dataset feature	f	Dataset feature
1	`zlogz`	9	`d2_a1_b1_mmdt`
2	`c1_b0_mmdt`	10	`d2_a1_b2_mmdt`
3	`c1_b1_mmdt`	11	`m2_b1_mmdt`
4	`c1_b2_mmdt`	12	`m2_b2_mmdt`
5	`c2_b1_mmdt`	13	`n2_b1_mmdt`
6	`c2_b2_mmdt`	14	`n2_b2_mmdt`
7	`d2_b1_mmdt`	15	`mass_mmdt`
8	`d2_b2_mmdt`	16	`multiplicity`

Implementation supplement: complete forest printout

Implementation and reproducibility

This record is generated by `TreePrinter.format_tree` from the exact v3 standardized model. It lists every internal decision and every leaf output in preorder, with heap indices and root-to-node paths. A node labelled unreachable is excluded only by an input-independent constant ancestor, not by absent training or validation examples. Numeric values are displayed with nine significant digits. The full 17-digit record is checked in as `generated/compact_forest_full_precision.txt`. These listings are for inspection; implementation should consume the frozen JSON or hexadecimal header. In all trees, a leaf output contributes to the background logit; the signal margin is the *negative sum* of the three selected outputs.

Reading the record. A path such as `RLLR` means right, left, left, right from the root. This path is the scientific node or leaf label. The extra `N` and `L` labels are internal-node and leaf-array indices; `heap` is a storage index. They have no mathematical role. Trees are numbered from zero in the file and from one in the scientific text.

Implementation and reproducibility

Tree 0 (scientific tree $t = 1$).

```
TREE 0: depth=4, leaves=linear, input=standardized
Coefficient display: 9 significant digits; exact zeros omitted.
At every cut: h > 0 goes RIGHT; h <= 0 goes LEFT (including equality).

N0 [path=root; heap=0; reachable]
  h_0_0(x) = 1 * c1_b2_mmdt + 0.421620399
  <= 0 -> N1; > 0 -> N2

  N1 [path=L; heap=1; reachable]
    h_0_1(x) = 0
    <= 0 -> N3; > 0 -> N4; CONSTANT: always LEFT

    N3 [path=LL; heap=3; reachable]
      h_0_3(x) = 0
      <= 0 -> N7; > 0 -> N8; CONSTANT: always LEFT

      N7 [path=LLL; heap=7; reachable]
        h_0_7(x) = 0.318308228 * c1_b0_mmdt + 0.330704464
        <= 0 -> L0; > 0 -> L1

        L0 [path=LLLL; heap=15; reachable]
          f_0_0(x) = 0.723115375 * zlogz - 0.424343964 * c1_b0_mmdt - 0.446947898 * c1_b1_mmdt -
↪ 0.415001941 * c2_b1_mmdt - 0.139216682 * d2_b1_mmdt - 0.139216682 * d2_a1_b1_mmdt + 0.54944101 *
↪ m2_b1_mmdt - 0.495073946 * m2_b2_mmdt + 0.059415836 * n2_b1_mmdt + 0.858013396 * multiplicity -
↪ 1.35727329

        L1 [path=LLLR; heap=16; reachable]
          f_0_1(x) = 1.51929697 * zlogz + 0.669978191 * c1_b0_mmdt - 3.56315299 * c1_b2_mmdt +
↪ 0.613457803 * c2_b1_mmdt - 0.396278128 * d2_b1_mmdt + 0.152304141 * d2_b2_mmdt - 0.396278128 *
↪ d2_a1_b1_mmdt - 0.199974866 * m2_b2_mmdt + 0.980332415 * n2_b1_mmdt - 0.420497004 * n2_b2_mmdt +
↪ 0.396719294 * mass_mmdt + 1.34144785 * multiplicity - 1.72085229

      N8 [path=LLR; heap=8; UNREACHABLE (constant ancestor)]
        h_0_8(x) = 0
        <= 0 -> L2; > 0 -> L3; CONSTANT: always LEFT

        L2 [path=LLRL; heap=17; UNREACHABLE (constant ancestor)]
          f_0_2(x) = 0

        L3 [path=LLRR; heap=18; UNREACHABLE (constant ancestor)]
          f_0_3(x) = 0

    N4 [path=LR; heap=4; UNREACHABLE (constant ancestor)]
      h_0_4(x) = 0
      <= 0 -> N9; > 0 -> N10; CONSTANT: always LEFT

      N9 [path=LRL; heap=9; UNREACHABLE (constant ancestor)]
        h_0_9(x) = -0.0491155824 * c1_b0_mmdt + 0.000379329885
        <= 0 -> L4; > 0 -> L5

        L4 [path=LRLL; heap=19; UNREACHABLE (constant ancestor)]
          f_0_4(x) = 0

        L5 [path=LRLR; heap=20; UNREACHABLE (constant ancestor)]
          f_0_5(x) = 0
```

```
      N10 [path=LRR; heap=10; UNREACHABLE (constant ancestor)]
        h_0_10(x) = 1.75931373e-09 * c1_b0_mmdt - 3.92622475e-09 * m2_b2_mmdt + 1.22858604e-07 *
↪ multiplicity - 1.097723e-07
        <= 0 -> L6; > 0 -> L7
        L6 [path=LRRL; heap=21; UNREACHABLE (constant ancestor)]
          f_0_6(x) = 0
        L7 [path=LRRR; heap=22; UNREACHABLE (constant ancestor)]
          f_0_7(x) = 0
  N2 [path=R; heap=2; reachable]
    h_0_2(x) = 0.0881065951 * c1_b2_mmdt + 0.0025365055 * c2_b2_mmdt + 0.17735909 * m2_b1_mmdt +
↪ 0.0312233619 * mass_mmdt + 0.100640807 * multiplicity + 0.0190586642
    <= 0 -> N5; > 0 -> N6
    N5 [path=RL; heap=5; reachable]
      h_0_5(x) = 0
      <= 0 -> N11; > 0 -> N12; CONSTANT: always LEFT
      N11 [path=RLL; heap=11; reachable]
        h_0_11(x) = -9.45167785e-16 * d2_b1_mmdt - 5.84426162e-16
        <= 0 -> L8; > 0 -> L9
        L8 [path=RLLL; heap=23; reachable]
          f_0_8(x) = 0.472450802 * zlogz + 0.0989313467 * c1_b0_mmdt - 0.831430736 * c1_b2_mmdt -
↪ 0.431555721 * c2_b2_mmdt + 0.0150358961 * d2_b1_mmdt + 0.648844514 * d2_b2_mmdt + 0.0150358961 *
↪ d2_a1_b1_mmdt - 3.09641516 * d2_a1_b2_mmdt + 1.49849996 * m2_b1_mmdt + 1.81153643 * n2_b1_mmdt -
↪ 0.893777123 * n2_b2_mmdt + 0.182450792 * multiplicity - 2.83938374
        L9 [path=RLLR; heap=24; reachable]
          f_0_9(x) = 0.232327574 * zlogz - 0.23205911 * c1_b0_mmdt - 0.516471161 * c1_b1_mmdt -
↪ 0.848527306 * c2_b1_mmdt - 1.97477014 * c2_b2_mmdt + 1.42114491 * d2_b2_mmdt + 0.536602443 *
↪ m2_b1_mmdt + 0.651756707 * mass_mmdt - 1.64666395
      N12 [path=RLR; heap=12; UNREACHABLE (constant ancestor)]
        h_0_12(x) = 9.89173599e-11 * c1_b2_mmdt + 1.79437153e-09 * c2_b1_mmdt + 1.81919634e-09 *
↪ c2_b2_mmdt + 1.04854961e-09
        <= 0 -> L10; > 0 -> L11
        L10 [path=RLRL; heap=25; UNREACHABLE (constant ancestor)]
          f_0_10(x) = 0
        L11 [path=RLRR; heap=26; UNREACHABLE (constant ancestor)]
          f_0_11(x) = 0
    N6 [path=RR; heap=6; reachable]
      h_0_6(x) = 0.164748862 * c1_b2_mmdt + 0.185195199 * c2_b2_mmdt + 0.271888733 * d2_a1_b2_mmdt +
↪ 1.06018366 * mass_mmdt - 0.265110937 * multiplicity - 1.32289565
      <= 0 -> N13; > 0 -> N14
      N13 [path=RRL; heap=13; reachable]
        h_0_13(x) = 0.635416434 * c1_b2_mmdt - 0.475835509 * multiplicity + 0.00561843162
        <= 0 -> L12; > 0 -> L13
        L12 [path=RRLL; heap=27; reachable]
          f_0_12(x) = 0.000756449729 * zlogz + 0.576353531 * c1_b0_mmdt + 1.62064775 * c1_b1_mmdt -
↪ 2.48681119 * c1_b2_mmdt - 2.10518311 * c2_b1_mmdt + 1.37970957 * c2_b2_mmdt - 0.0135562553 *
↪ d2_b1_mmdt + 0.338362995 * d2_b2_mmdt - 0.0135562553 * d2_a1_b1_mmdt - 1.72149735 * d2_a1_b2_mmdt -
↪ 0.38025814 * m2_b1_mmdt - 0.351321056 * m2_b2_mmdt + 2.69042188 * n2_b1_mmdt - 0.919390109 *
↪ n2_b2_mmdt + 0.32251432 * mass_mmdt + 0.582538651 * multiplicity - 3.45473882
        L13 [path=RRLR; heap=28; reachable]
          f_0_13(x) = -0.140117186 * c1_b2_mmdt + 0.652505422 * c2_b1_mmdt + 0.253434919 * d2_b1_mmdt -
↪ 0.602112106 * d2_b2_mmdt + 0.253434919 * d2_a1_b1_mmdt - 0.527210682 * d2_a1_b2_mmdt - 0.00193453091 *
↪  n2_b2_mmdt - 1.64156297 * mass_mmdt + 0.0192123329 * multiplicity - 2.68453555
      N14 [path=RRR; heap=14; reachable]
        h_0_14(x) = 0
        <= 0 -> L14; > 0 -> L15; CONSTANT: always LEFT
        L14 [path=RRRL; heap=29; reachable]
          f_0_14(x) = -0.727395772 * zlogz - 1.0021119 * c1_b0_mmdt - 1.77913852 * c1_b1_mmdt +
↪ 1.29863387 * c1_b2_mmdt - 5.22419531 * c2_b1_mmdt + 1.67449112 * c2_b2_mmdt + 0.609159654 *
↪ d2_b1_mmdt - 1.14240277 * d2_b2_mmdt + 0.609159654 * d2_a1_b1_mmdt + 0.462711749 * d2_a1_b2_mmdt -
↪ 0.379623445 * m2_b1_mmdt - 1.49230987 * m2_b2_mmdt + 1.44153754 * n2_b1_mmdt - 4.67940637 * mass_mmdt
↪  + 1.29820647 * multiplicity + 5.68806483
        L15 [path=RRRR; heap=30; UNREACHABLE (constant ancestor)]
          f_0_15(x) = 0
Selected leaf f is a class-0 logit contribution; forest signal margin = -sum(f).
```

Implementation and reproducibility

Tree 1 (scientific tree $t = 2$).

```
TREE 1: depth=4, leaves=linear, input=standardized
Coefficient display: 9 significant digits; exact zeros omitted.
At every cut: h > 0 goes RIGHT; h <= 0 goes LEFT (including equality).

N0 [path=root; heap=0; reachable]
  h_1_0(x) = 1 * c1_b2_mmdt + 0.421515048
  <= 0 -> N1; > 0 -> N2

  N1 [path=L; heap=1; reachable]
    h_1_1(x) = -2.3776136e-11 * multiplicity - 3.17032814e-11
    <= 0 -> N3; > 0 -> N4

    N3 [path=LL; heap=3; reachable]
      h_1_3(x) = -0.0032891119 * zlogz + 0.0142175742 * d2_b1_mmdt + 0.00121827303 * d2_a1_b1_mmdt -
↪ 0.000244599886 * m2_b2_mmdt - 0.00553888952 * n2_b2_mmdt + 0.295132576 * mass_mmdt + 0.0218087714
      <= 0 -> N7; > 0 -> N8

      N7 [path=LLL; heap=7; reachable]
        h_1_7(x) = 0.0213173363 * zlogz + 0.612901764 * c1_b0_mmdt - 0.0606512036
        <= 0 -> L0; > 0 -> L1

        L0 [path=LLLL; heap=15; reachable]
          f_1_0(x) = 1.02493155 * zlogz + 0.475780074 * c1_b0_mmdt - 3.46865641 * c1_b1_mmdt +
↪ 1.39660226 * c1_b2_mmdt + 0.0808524935 * d2_b1_mmdt + 0.0625866107 * d2_b2_mmdt + 0.0808524935 *
↪ d2_a1_b1_mmdt + 0.265262813 * d2_a1_b2_mmdt + 0.0469595498 * m2_b1_mmdt - 0.00734786374 * m2_b2_mmdt +
↪  3.49945192 * mass_mmdt - 0.346146245

        L1 [path=LLLR; heap=16; reachable]
          f_1_1(x) = 2.42880675 * zlogz + 3.52193453 * c1_b0_mmdt - 2.27198818 * c1_b1_mmdt - 0.640806627
↪  * c1_b2_mmdt + 0.0986683506 * d2_b1_mmdt + 0.101873458 * d2_b2_mmdt + 0.0986683506 * d2_a1_b1_mmdt +
↪  0.331194728 * m2_b1_mmdt + 0.0539714566 * n2_b1_mmdt + 3.24454093 * mass_mmdt + 0.503474228 *
↪ multiplicity - 1.50898053

      N8 [path=LLR; heap=8; reachable]
        h_1_8(x) = 7.32169007e-19 * zlogz - 2.06778204e-19 * d2_b1_mmdt - 1.91895639e-18 * d2_a1_b1_mmdt
↪  + 4.16333634e-17 * mass_mmdt + 5.02329003e-19
        <= 0 -> L2; > 0 -> L3

        L2 [path=LLRL; heap=17; reachable]
          f_1_2(x) = 0.398116558 * d2_b1_mmdt + 0.398116558 * d2_a1_b1_mmdt - 0.660830844 *
↪ d2_a1_b2_mmdt - 0.114827421 * m2_b2_mmdt - 0.310979867 * n2_b2_mmdt + 0.846928396 * multiplicity -
↪ 0.9437156

        L3 [path=LLRR; heap=18; reachable]
          f_1_3(x) = 0.244294729 * zlogz + 0.880510729 * d2_b2_mmdt - 1.42081066 * m2_b1_mmdt +
↪ 1.71265082 * multiplicity - 2.14718376

    N4 [path=LR; heap=4; reachable]
      h_1_4(x) = 0
      <= 0 -> N9; > 0 -> N10; CONSTANT: always LEFT

      N9 [path=LRL; heap=9; reachable]
        h_1_9(x) = 0.128095519 * m2_b1_mmdt + 0.00063403479 * multiplicity - 0.0832689733
        <= 0 -> L4; > 0 -> L5

        L4 [path=LRLL; heap=19; reachable]
          f_1_4(x) = 0.232595756 * zlogz + 0.336338146 * c1_b0_mmdt + 0.186230458 * d2_b1_mmdt +
↪ 0.186230458 * d2_a1_b1_mmdt - 0.0975771333 * m2_b1_mmdt + 0.100432843 * m2_b2_mmdt - 0.712895383 *
↪ multiplicity - 1.51109101

        L5 [path=LRLR; heap=20; reachable]
          f_1_5(x) = 0.26497515 * d2_b2_mmdt + 0.322656527 * n2_b2_mmdt - 0.656358636

      N10 [path=LRR; heap=10; UNREACHABLE (constant ancestor)]
        h_1_10(x) = 3.11856008e-17 * zlogz + 1.03393415e-17 * d2_b1_mmdt + 3.62950285e-17 *
↪ d2_a1_b1_mmdt - 3.214346e-17 * m2_b1_mmdt - 5.71317478e-17 * m2_b2_mmdt + 5.83355278e-17 * mass_mmdt +
↪  1.70316265e-11 * multiplicity - 1.89719708e-11
        <= 0 -> L6; > 0 -> L7

        L6 [path=LRRL; heap=21; UNREACHABLE (constant ancestor)]
          f_1_6(x) = 0

        L7 [path=LRRR; heap=22; UNREACHABLE (constant ancestor)]
          f_1_7(x) = 0

  N2 [path=R; heap=2; reachable]
    h_1_2(x) = 1.08273826 * c1_b2_mmdt + 0.24935691 * c2_b2_mmdt - 0.0971598576 * d2_b1_mmdt +
↪ 0.256354852 * d2_b2_mmdt - 0.0929252735 * d2_a1_b1_mmdt - 0.228292085 * m2_b1_mmdt + 0.0567631958 *
↪ mass_mmdt + 0.28027011 * multiplicity - 0.13138567
    <= 0 -> N5; > 0 -> N6
```

```
    N5 [path=RL; heap=5; reachable]
      h_1_5(x) = 0
      <= 0 -> N11; > 0 -> N12; CONSTANT: always LEFT
      N11 [path=RLL; heap=11; reachable]
        h_1_11(x) = -0.259890766 * d2_a1_b2_mmdt + 0.208572797 * m2_b1_mmdt + 3.31046124 * mass_mmdt -
↪ 0.0387863599
        <= 0 -> L8; > 0 -> L9
        L8 [path=RLLL; heap=23; reachable]
          f_1_8(x) = 1.49591861 * zlogz + 0.441575961 * c1_b0_mmdt + 1.71623752 * c1_b2_mmdt +
↪ 0.707286758 * d2_a1_b2_mmdt + 0.136799401 * n2_b1_mmdt - 0.0922364294 * n2_b2_mmdt - 4.1902612 *
↪ mass_mmdt + 0.994771372 * multiplicity + 0.902352018
        L9 [path=RLLR; heap=24; reachable]
          f_1_9(x) = 1.06616852 * zlogz + 0.26139261 * c1_b0_mmdt - 1.59073627 * c1_b1_mmdt - 3.31259981
↪  * c2_b1_mmdt + 2.90835606 * c2_b2_mmdt + 0.0402529973 * d2_b1_mmdt + 0.0402529973 * d2_a1_b1_mmdt +
↪ 3.30625 * d2_a1_b2_mmdt - 2.78491928 * m2_b1_mmdt + 0.342727361 * m2_b2_mmdt - 0.69660259 *
↪ n2_b2_mmdt + 1.00497391 * mass_mmdt + 2.54220983 * multiplicity + 0.0829818883
      N12 [path=RLR; heap=12; UNREACHABLE (constant ancestor)]
        h_1_12(x) = 3.32650029e-05 * c1_b2_mmdt + 1.11948505e-05 * c2_b2_mmdt + 2.49204468e-05 *
↪ d2_a1_b2_mmdt + 1.99509567e-06 * m2_b2_mmdt + 0.000137262248 * n2_b1_mmdt + 9.50083532e-05
        <= 0 -> L10; > 0 -> L11
        L10 [path=RLRL; heap=25; UNREACHABLE (constant ancestor)]
          f_1_10(x) = 0
        L11 [path=RLRR; heap=26; UNREACHABLE (constant ancestor)]
          f_1_11(x) = 0
    N6 [path=RR; heap=6; reachable]
      h_1_6(x) = 4.94065646e-324
      <= 0 -> N13; > 0 -> N14; CONSTANT: always RIGHT
      N13 [path=RRL; heap=13; UNREACHABLE (constant ancestor)]
        h_1_13(x) = -2.68948213e-10 * zlogz - 5.82051432e-11 * c2_b1_mmdt - 2.85373406e-10 * d2_b1_mmdt +
↪  2.05721037e-10 * d2_b2_mmdt - 1.06470833e-10 * d2_a1_b1_mmdt + 9.05405039e-12 * d2_a1_b2_mmdt +
↪ 3.42554664e-10 * m2_b2_mmdt + 2.13357549e-10 * n2_b2_mmdt - 8.89638165e-10 * mass_mmdt + 1.4924407e
↪ -09 * multiplicity - 1.72964941e-09
        <= 0 -> L12; > 0 -> L13
        L12 [path=RRLL; heap=27; UNREACHABLE (constant ancestor)]
          f_1_12(x) = 0
        L13 [path=RRLR; heap=28; UNREACHABLE (constant ancestor)]
          f_1_13(x) = 0
      N14 [path=RRR; heap=14; reachable]
        h_1_14(x) = -0.0357752902 * c1_b2_mmdt + 1.00738533 * mass_mmdt - 1.85526559
        <= 0 -> L14; > 0 -> L15
        L14 [path=RRRL; heap=29; reachable]
          f_1_14(x) = 0.992368325 * zlogz - 4.80052499 * c1_b1_mmdt + 2.90036981 * c1_b2_mmdt -
↪ 0.714839331 * c2_b1_mmdt - 0.349819173 * d2_b1_mmdt - 0.0440891415 * d2_b2_mmdt - 0.349819173 *
↪ d2_a1_b1_mmdt + 0.777214702 * d2_a1_b2_mmdt + 0.176693713 * m2_b1_mmdt + 2.01859804 * n2_b1_mmdt -
↪ 1.16458901 * n2_b2_mmdt - 1.53336443 * mass_mmdt + 0.283339646 * multiplicity + 2.63807278
        L15 [path=RRRR; heap=30; reachable]
          f_1_15(x) = -0.485616316 * c1_b0_mmdt - 0.625011056 * c2_b2_mmdt + 0.51939675 * d2_b1_mmdt +
↪ 0.199901858 * d2_b2_mmdt + 0.51939675 * d2_a1_b1_mmdt - 0.491209214 * m2_b2_mmdt + 2.38441955 *
↪ mass_mmdt - 0.280424752 * multiplicity - 5.0660948
Selected leaf f is a class-0 logit contribution; forest signal margin = -sum(f).
```

Implementation and reproducibility

Tree 2 (scientific tree $t = 3$).

```
TREE 2: depth=4, leaves=linear, input=standardized
Coefficient display: 9 significant digits; exact zeros omitted.
At every cut: h > 0 goes RIGHT; h <= 0 goes LEFT (including equality).
N0 [path=root; heap=0; reachable]
  h_2_0(x) = 0.451963069 * c1_b2_mmdt - 0.00922949319 * m2_b1_mmdt + 0.315856319 * mass_mmdt +
↪ 0.0990362496
  <= 0 -> N1; > 0 -> N2
  N1 [path=L; heap=1; reachable]
    h_2_1(x) = -0.0393006101 * zlogz + 0.0190956747 * multiplicity + 0.00401865025
    <= 0 -> N3; > 0 -> N4
```

```
    N3 [path=LL; heap=3; reachable]
      h_2_3(x) = 0
      <= 0 -> N7; > 0 -> N8; CONSTANT: always LEFT
      N7 [path=LLL; heap=7; reachable]
        h_2_7(x) = -0.338370941 * zlogz - 0.197456658 * c1_b0_mmdt + 0.505337081 * multiplicity +
↪ 0.323631119
        <= 0 -> L0; > 0 -> L1
        L0 [path=LLLL; heap=15; reachable]
          f_2_0(x) = 0.391007555 * c1_b0_mmdt + 2.7854501 * c1_b1_mmdt - 5.98969282 * c1_b2_mmdt +
↪ 0.733907917 * c2_b1_mmdt - 0.0568167023 * d2_b2_mmdt - 0.461051796 * m2_b1_mmdt + 0.358625148 *
↪ m2_b2_mmdt + 0.00378192334 * n2_b2_mmdt - 2.04581277 * mass_mmdt - 0.432532063 * multiplicity -
↪ 1.93961875
        L1 [path=LLLR; heap=16; reachable]
          f_2_1(x) = -0.441179138 * zlogz - 3.22570484 * c1_b2_mmdt - 0.0681594455 * d2_b1_mmdt -
↪ 0.0681594455 * d2_a1_b1_mmdt + 0.311882288 * d2_a1_b2_mmdt - 0.131522611 * n2_b1_mmdt - 0.234805732 *
↪  mass_mmdt + 0.648265323 * multiplicity - 0.763080965
      N8 [path=LLR; heap=8; UNREACHABLE (constant ancestor)]
        h_2_8(x) = 0.00394358152 * mass_mmdt - 0.000176803622
        <= 0 -> L2; > 0 -> L3
        L2 [path=LLRL; heap=17; UNREACHABLE (constant ancestor)]
          f_2_2(x) = 0
        L3 [path=LLRR; heap=18; UNREACHABLE (constant ancestor)]
          f_2_3(x) = 0
    N4 [path=LR; heap=4; reachable]
      h_2_4(x) = 0.748689614 * c1_b0_mmdt + 0.669457034 * c2_b1_mmdt + 0.101051343
      <= 0 -> N9; > 0 -> N10
      N9 [path=LRL; heap=9; reachable]
        h_2_9(x) = 0
        <= 0 -> L4; > 0 -> L5; CONSTANT: always LEFT
        L4 [path=LRLL; heap=19; reachable]
          f_2_4(x) = -0.389784207 * zlogz - 0.209597106 * c1_b0_mmdt - 3.08379498 * c1_b1_mmdt -
↪ 1.4593788 * c2_b1_mmdt + 0.0255711958 * d2_b1_mmdt - 0.0574990566 * d2_b2_mmdt + 0.0255711958 *
↪ d2_a1_b1_mmdt + 0.973389446 * d2_a1_b2_mmdt + 0.250244728 * m2_b2_mmdt - 0.782019914 * n2_b1_mmdt +
↪ 0.0194565833 * n2_b2_mmdt + 2.61367386 * mass_mmdt + 0.993769728 * multiplicity + 0.228651881
        L5 [path=LRLR; heap=20; UNREACHABLE (constant ancestor)]
          f_2_5(x) = 0
      N10 [path=LRR; heap=10; reachable]
        h_2_10(x) = -0.615381338 * zlogz - 0.780626703 * multiplicity - 0.248174582
        <= 0 -> L6; > 0 -> L7
        L6 [path=LRRL; heap=21; reachable]
          f_2_6(x) = 0.405855831 * zlogz + 1.19454522 * c1_b0_mmdt - 0.171973875 * c1_b1_mmdt -
↪ 3.01430074 * c2_b1_mmdt + 1.66035781 * c2_b2_mmdt + 0.195722301 * d2_b2_mmdt - 0.470821689 *
↪ d2_a1_b2_mmdt + 0.725279387 * m2_b1_mmdt - 0.0949253938 * m2_b2_mmdt + 1.53492 * n2_b1_mmdt -
↪ 0.872843694 * n2_b2_mmdt + 1.27018591 * mass_mmdt + 0.644659641 * multiplicity + 0.542419575
        L7 [path=LRRR; heap=22; reachable]
          f_2_7(x) = 3.48806978 * zlogz + 7.81025712 * c1_b0_mmdt - 0.95346229 * c1_b2_mmdt +
↪ 0.280675132 * c2_b1_mmdt + 0.0334405957 * d2_b2_mmdt - 1.01384781 * d2_a1_b2_mmdt + 1.85257453 *
↪ m2_b1_mmdt - 0.533779879 * m2_b2_mmdt - 0.647558288 * n2_b1_mmdt + 0.591781799 * n2_b2_mmdt -
↪ 1.08156364 * mass_mmdt + 1.20551885 * multiplicity - 2.42799078
  N2 [path=R; heap=2; reachable]
    h_2_2(x) = 0.168562252 * c2_b2_mmdt - 0.0982298021 * d2_a1_b2_mmdt + 0.37090396 * m2_b1_mmdt +
↪ 0.113712418 * multiplicity - 0.745275257
    <= 0 -> N5; > 0 -> N6
    N5 [path=RL; heap=5; reachable]
      h_2_5(x) = 0.814138721 * mass_mmdt + 0.0531775224 * multiplicity - 0.29709854
      <= 0 -> N11; > 0 -> N12
      N11 [path=RLL; heap=11; reachable]
        h_2_11(x) = 4.94065646e-324
        <= 0 -> L8; > 0 -> L9; CONSTANT: always RIGHT
        L8 [path=RLLL; heap=23; UNREACHABLE (constant ancestor)]
          f_2_8(x) = 0
        L9 [path=RLLR; heap=24; reachable]
          f_2_9(x) = -0.273060581 * zlogz + 0.214560559 * c1_b0_mmdt + 2.79082823 * c1_b2_mmdt +
↪ 0.0872642558 * d2_b1_mmdt + 0.0872642558 * d2_a1_b1_mmdt + 0.0668304306 * d2_a1_b2_mmdt + 0.700471572
↪  * m2_b2_mmdt - 0.377933505 * n2_b2_mmdt + 0.446788216 * multiplicity + 0.784109979
```

```
      N12 [path=RLR; heap=12; reachable]
        h_2_12(x) = 2.06493224e-10 * c1_b2_mmdt + 6.8828261e-11 * c2_b1_mmdt + 2.89598635e-11 *
↪ m2_b1_mmdt + 5.35347149e-11 * m2_b2_mmdt - 1.75510092e-10
        <= 0 -> L10; > 0 -> L11
        L10 [path=RLRL; heap=25; reachable]
          f_2_10(x) = 0.0172917486 * zlogz + 0.479280898 * c1_b0_mmdt + 0.895727863 * c1_b1_mmdt -
↪ 2.47138391 * c1_b2_mmdt + 1.46797745 * c2_b1_mmdt + 0.0674431371 * d2_b1_mmdt - 0.20536907 *
↪ d2_b2_mmdt + 0.0674431371 * d2_a1_b1_mmdt + 0.960087986 * m2_b2_mmdt - 0.399753552 * n2_b1_mmdt -
↪ 0.688302486 * n2_b2_mmdt + 5.79450008 * mass_mmdt + 0.0327603624 * multiplicity - 0.350562803
        L11 [path=RLRR; heap=26; reachable]
          f_2_11(x) = 0.851042201 * c1_b0_mmdt + 1.12087708 * c1_b2_mmdt - 2.14848846 * c2_b1_mmdt +
↪ 0.740207807 * d2_b1_mmdt + 0.740207807 * d2_a1_b1_mmdt + 1.30902686 * m2_b1_mmdt + 4.77523926 *
↪ mass_mmdt + 1.62965024
    N6 [path=RR; heap=6; reachable]
      h_2_6(x) = 0
      <= 0 -> N13; > 0 -> N14; CONSTANT: always LEFT
      N13 [path=RRL; heap=13; reachable]
        h_2_13(x) = 4.94065646e-324
        <= 0 -> L12; > 0 -> L13; CONSTANT: always RIGHT
        L12 [path=RRLL; heap=27; UNREACHABLE (constant ancestor)]
          f_2_12(x) = 0
        L13 [path=RRLR; heap=28; reachable]
          f_2_13(x) = -0.148614563 * zlogz + 1.11851788 * c1_b0_mmdt + 0.96743728 * c1_b1_mmdt +
↪ 0.00568823559 * c2_b2_mmdt + 0.40419135 * d2_b2_mmdt - 1.01497415 * d2_a1_b2_mmdt + 1.11598924 *
↪ m2_b1_mmdt + 1.21950507 * n2_b1_mmdt - 0.144073893 * n2_b2_mmdt + 4.08231683 * mass_mmdt -
↪ 0.113704284 * multiplicity - 0.734827284
      N14 [path=RRR; heap=14; UNREACHABLE (constant ancestor)]
        h_2_14(x) = 0.078412842 * c1_b2_mmdt - 0.128921279
        <= 0 -> L14; > 0 -> L15
        L14 [path=RRRL; heap=29; UNREACHABLE (constant ancestor)]
          f_2_14(x) = 0
        L15 [path=RRRR; heap=30; UNREACHABLE (constant ancestor)]
          f_2_15(x) = 0
Selected leaf f is a class-0 logit contribution; forest signal margin = -sum(f).
```